UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi
Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS 216 Homework Assignment

- Date Issued: **26 February 2026**
- Date Due: **18 May 2026** before **08:00**
- Submission Procedure: **Upload to ClickUP. An assignment upload will be made available.**
- Submission Format: **zip or tar + gzip archive**
- This assignment consists of **3 tasks** for a total of **145 marks**.

NB: Please read through the entire specification before asking questions on Discord/Email, as there is a high likelihood your question may be answered later in the specification.

1 Introduction

During this homework assignment, you will be implementing WebSockets to create a real-time flight tracking system. The system will allow Passengers to view their upcoming and current flights in real time, while Air Traffic Controllers (ATC) can manage and dispatch flights. This system will contain a map and use GPS coordinates (longitude and latitude in Decimal Degrees) to show the live position of aircraft between airports. More details will be provided below.

On completion, your assignment must contain the following:

- A PHP API hosted on Wheatley that stores and retrieves flight data from a MySQL Database.
- A **local** NodeJS WebSocket server that polls your PHP API on Wheatley and manages real-time communication between all connected clients.
- An Angular web client that connects to your NodeJS WebSocket server that will act as the frontend.

2 Constraints

1. You may complete this assignment individually or in groups of 3 (register on CS Portal).
2. You may ask the Teaching Assistants for help, but they will not be able to give you the solutions.
3. You must produce all of the source files yourself; you may not use any tool to generate source files or fragments thereof automatically. **This includes ChatGPT, Gemini etc!**
4. All written code should contain comments including your names, surnames and student numbers at the top of each file.
5. Your assignment must be programmed in NodeJS, HTML, JS, CSS and PHP. (Bootstrap is allowed).

6. You are required to use Angular 17 for the frontend client.
7. The API and Database should be hosted off Wheatley, the NodeJS server should be hosted locally.

3 Submission Instructions

You are required to upload all your source files and a README in an archive, either zipped or tar gzipped, to **clickUP**. No late submissions will be accepted (See due date and time at the top of the document), so make sure you upload in good time. Only one member is required to upload the files for the group.

NB: Ensure that you do not submit your code with your passwords or credentials included in your clickUP submission. Sanitise your files before uploading.

4 Resources

NodeJS - https://www.w3schools.com/nodejs/
https://www.tutorialspoint.com/nodejs/index.htm

ExpressJS - https://www.tutorialspoint.com/expressjs/index.htm
https://expressjs.com/
https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs

Web Sockets - https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
https://en.wikipedia.org/wiki/WebSocket
https://github.com/websockets/ws
https://socket.io/
https://www.npmjs.com/package/websocket

Leaflet https://leafletjs.com/index.html

Angular https://v17.angular.io/docs

Great-circle interpolation (linear interpolation will be sufficient for this practical) https://www.movable-type.co.uk/scripts/latlong.html

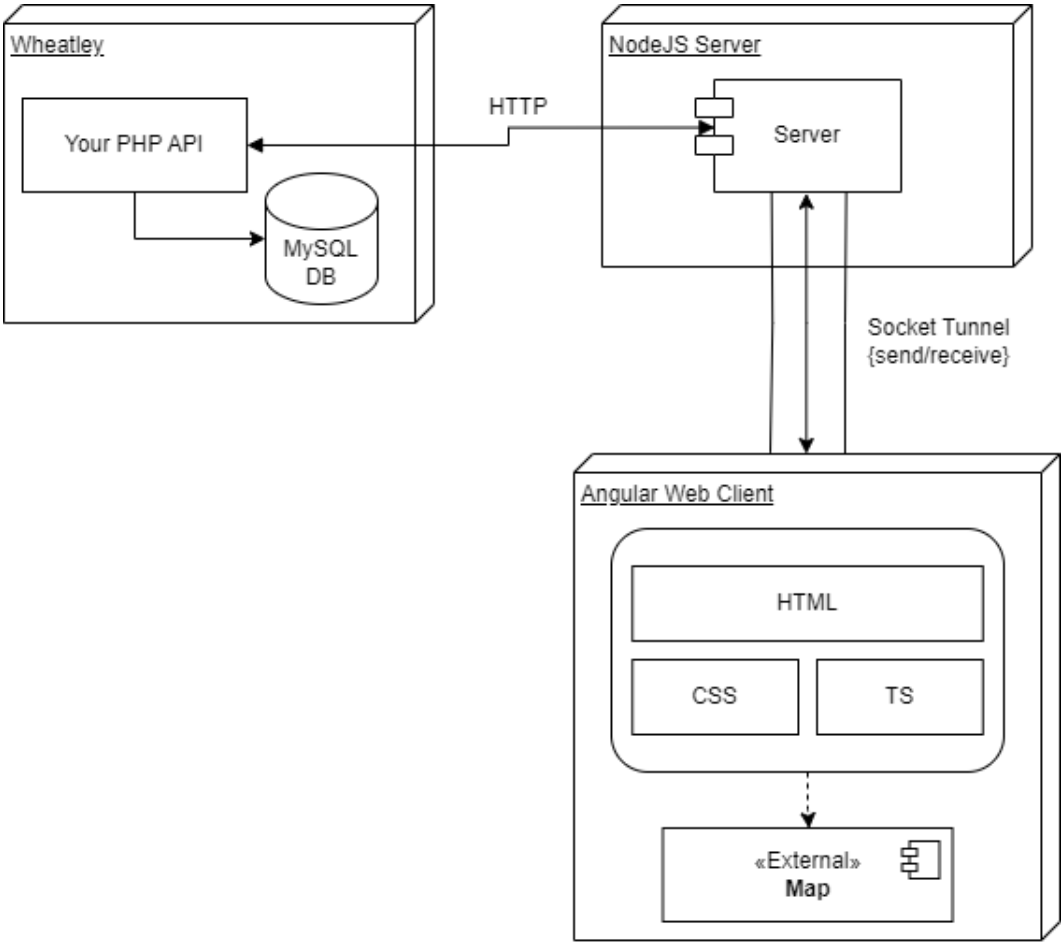
DOM Manipulation https://www.w3schools.com/js/js_htmldom.asp

5 Rubric for marking

Task 1 - PHP API & DB	**50**
MySQL DB	5
Login endpoint	5
GetAllFlights endpoint	8
GetFlight endpoint	5
DispatchFlight endpoint	8
UpdateFlightPosition endpoint	7
GetAirports endpoint	7
BoardFlight endpoint	5
Task 2 - Multi-User NodeJS WebSocket Server	**45**
Port can be chosen at run-time; reserved ports blocked	5
Server can accept multiple clients simultaneously	5
ATC client can dispatch a flight via UI; server animates and notifies passengers	8
Passenger client can confirm boarding via UI; server enforces 60-second window	5
`TRACK` message correctly subscribes client to live position updates	5
`KILL` CLI command is implemented and fully functioning	4
`QUIT` CLI command is implemented and fully functioning	4
If the connection is closed on the client, the server closes the socket	5
Server implements the full system flow correctly	2
ReadMe explanation, Update Every Step vs Update on Interval	2
Task 3 - Angular Web Client	**50**
Login functionality (WebSocket-based authentication flow)	5
Role-based interface (Passenger vs ATC views)	5
Leaflet map correctly initialised and styled	5
Airports plotted correctly using API data	5
Flight marker animation (real-time interpolation updates)	10
DISPATCH notification handling + 60-second boarding UI	5
TRACK subscription handling	5
Error handling (403, 404, timeout, socket disconnect etc.)	5
Clean UI/UX and correct system flow implementation	5
Upload	
Not uploaded to clickUP	**-145**
Bonus	**15**
Total	**145**

6 Overview

To understand the architecture of this assignment, please consult the diagram below.



7 Description of Flight Tracking System

In this assignment, you are implementing a real-time flight tracking system. The system allows passengers to monitor their upcoming and active flights, while Air Traffic Controllers can manage and dispatch flights in real time. An animated aircraft marker will fly between origin and destination airports on a Leaflet map for all users to observe.

Note: To make flights feasible to demonstrate, the server must scale flight durations as follows: a flight with a real-world duration of N hours must complete its full animation in N *seconds*. For example, a 14-hour flight must animate in 14 seconds.

This system contains two types of users:

- **Passenger**: A registered user who has one or more upcoming flights. Passengers can view their own flight schedule and track their current flight in real time on the map.
- **ATC (Air Traffic Controller)**: An operator who can view all scheduled flights across all passengers, dispatch flights, and track the live position of any active aircraft.

7.1 Passenger

The passenger will be the user who views their upcoming flights and tracks their current flight. Once an ATC dispatches their flight, the passenger is notified and must confirm they are boarding within **60 seconds**. If the passenger does not confirm within this window, they are marked as a no-show for that flight.

7.2 ATC (Air Traffic Controller)

The ATC will be the user who views all scheduled flights across all passengers, selects a scheduled flight to dispatch via a button in the Angular client, and monitors live aircraft positions on the map. The ATC can track any active flight and is responsible for initiating the boarding process by dispatching a flight.

8 Assignment Instructions

Task 1: PHP API + Database ..... (50 marks)

You need to create a PHP API that builds on top of your PA3 API by adding flight tracking endpoints for this assignment. It will be hosted off Wheatley and will make use of a MySQL database. This assignment will require a database with four tables, namely `Users`, `Airports`, `Flights`, and `Passenger_Flights`. The `Users` table is carried over from PA3.

Users are required to log in before they can view or interact with any flights. Note that login functionality will already be implemented in PA3, so it won't carry much weight in the HA.

8.1 API:

Your PHP API will need to handle user login and six other endpoints, namely:
`GetAllFlights`, `GetFlight`, `DispatchFlight`, `UpdateFlightPosition`, `GetAirports`, and `BoardFlight`.

- `GetAllFlights` — Returns flights based on the role of the authenticated user.
  - When an **ATC** uses this endpoint, they receive *all* flights with their current status and GPS position.
  - When a **Passenger** uses this endpoint, they receive only the flights they are personally booked onto.
- `GetFlight` — Returns detailed information for a single flight.
  - **ATC**: Receives full flight detail, including the complete passenger list for that flight.
  - **Passenger**: Receives flight detail only if they are booked on it; no passenger list is returned.
- `DispatchFlight` — Transitions a flight from `Scheduled` to `Boarding`. **ATC only.**
  - Records the `dispatched_at` timestamp.
  - Returns `400` if the flight is not in the `Scheduled` state.
- `UpdateFlightPosition` — Updates the current GPS coordinates and status of an in-flight aircraft. Called by the NodeJS server periodically as it interpolates the aircraft's position.
  - Updates `current_latitude`, `current_longitude`, and `status` for the given flight.
  - This is a server-to-server endpoint; no user session is required. Use a shared API key or similar mechanism.
- `GetAirports` — Returns the full list of airports with their GPS coordinates, used by the frontend to plot markers on the Leaflet map.
- `BoardFlight` — Called when a Passenger confirms they are boarding in response to an ATC dispatch notification. **Passenger only.**
  - Sets `boarding_confirmed = 1` and records `confirmed_at` for the authenticated passenger on this flight.
  - Returns `400` if the confirmation window has expired (more than 60 seconds after dispatch).

8.2 Database:

You will need to manually populate at least **10 airports** and at least **10 mock flights** in your database. Flights and passenger bookings may be seeded manually for testing purposes.
The tables must have the following fields (however, you may add more fields if you want):

8.2.1 Users

This table is carried over from PA3. It will store user data:

- `id`
- `username`

- `password`
- `email`
- `type` (`Passenger` or `ATC`)

8.2.2 Airports

This table stores information about airports used when plotting flight routes on the map:

- `id`
- `name`
- `iata_code` (unique 3-letter code, e.g. `JNB`)
- `city`
- `country`
- `latitude`
- `longitude`

8.2.3 Flights

This table stores all scheduled and active flights. At least 10 mock flights must be seeded manually:

- `id`
- `flight_number` (unique, e.g. `SA203`)
- `origin_airport_id` (foreign key → `Airports.id`)
- `destination_airport_id` (foreign key → `Airports.id`)
- `departure_time`
- `flight_duration_hours` (e.g. `14.5`)
- `status` (`Scheduled`, `Boarding`, `In Flight`, or `Landed`)
- `current_latitude` (updated in real time; defaults to origin airport)
- `current_longitude` (updated in real time; defaults to origin airport)
- `dispatched_at` (timestamp set when ATC dispatches; nullable)

8.2.4 Passenger_Flights

This is a join table to represent the many-to-many relationship between passengers and their flights:

- `id`
- `passenger_id` (foreign key → `Users.id`)
- `flight_id` (foreign key → `Flights.id`)
- `seat_number` (nullable, e.g. `12A`)
- `boarding_confirmed` (`0` = not confirmed, `1` = confirmed)
- `confirmed_at` (timestamp of boarding confirmation; nullable)

Task 2: Multi-User NodeJS WebSocket Server ..... (45 marks)

For this task, you are required to create a **Multi-User** WebSocket server **(on localhost)**. The server must be coded in **NodeJS locally and not off Wheatley**. You are allowed to use libraries for this (e.g. `ws`, `socket.io`).

Note: Wheatley is a web server that follows the LAMP stack and installing other applications on it brings security issues. You must therefore use localhost for this since the NodeJS server needs to run on an open port, and it would be difficult to allow the server to be run on Wheatley, as the chances of students using the same port would be high. Also, this is done to avoid some students from intentionally and unintentionally performing port blocking. This is also not possible to do due to UP's firewall.

It is important to understand that this server has **two distinct types of interaction**, which must not be confused with one another:

- **Server CLI commands** — typed directly into the terminal by whoever is running the server process. These are administrative controls that operate independently of any connected client.
- **WebSocket message types** — JSON messages exchanged over the socket connection between the Angular client and the server. These are triggered automatically by the user clicking buttons in the UI; the user *never* types these manually into a terminal.

8.3 Server Configuration

- The server must accept a port number at run-time (via a command-line argument or interactive prompt). The ports you are allowed to use are from 1024–49151, make sure the user can only choose a port in this range. *If you are curious about why this is the case, you can read up about it at https://www.techtarget.com/searchnetworking/definition/port-number.*
- Block the user from attempting to open a reserved port (any port outside the range above).
- Accept multiple client connections and interact with the clients concurrently through sockets.
- **The server must utilise the PHP API you developed in Task 1.** You must call your API endpoints to get, update, and dispatch flights.

8.4 Server CLI Commands

These commands are typed into the server's terminal by the person running the server. They are **not** sent by the Angular client and are not part of the WebSocket protocol.

- `FLIGHT_STATUS <flight_id>` — Prints the current status of a flight directly to the server terminal.
  - Flight number and current status.
  - Current GPS coordinates (latitude, longitude).
  - Number of passengers who have confirmed boarding vs. total passengers booked.
  - Estimated time remaining until landing.
- `KILL <username>` — Forcibly closes the WebSocket connection of the specified user.
  - The server must maintain a mapping of username to socket so that the correct connection can be identified.
  - The server must notify the affected user before closing their connection.
- `QUIT` — Broadcasts a shutdown message to all connected clients, then gracefully closes all connections and terminates the server process.

8.5 WebSocket Message Protocol

The messages below define the communication between the Angular client and the server over the WebSocket connection. They are **triggered by user interactions in the UI** — for example, clicking a button. The user never types these into a terminal. Each message must follow the JSON format specified.

8.5.1 Messages sent from the Angular client to the server

- `DISPATCH` — Sent by an **ATC client** when the ATC clicks the "Dispatch Flight" button in the UI.
  - On receiving this message, the server must:
    - Call the `DispatchFlight` API endpoint to update the flight status to `Boarding`.
    - Begin the flight animation loop, interpolating the aircraft's GPS position between origin and destination over N seconds (where N is the flight duration in hours).
    - Broadcast a `BOARDING_CALL` notification to all Passengers booked on that flight. The **60-second boarding window** begins at this point.
- `BOARD` — Sent by a **Passenger client** when the Passenger clicks the "Confirm Boarding" button in the UI after receiving a `BOARDING_CALL` notification.
  - On receiving this message, the server must:
    - Call the `BoardFlight` API endpoint to record the boarding confirmation.
    - If the 60-second window has already expired, send an error back to the Passenger and notify the ATC of the no-show instead.
    - Otherwise, notify the ATC that this Passenger has confirmed boarding.
- `TRACK` — Sent by either an **ATC client** or a **Passenger client** when they navigate to the flight tracking view in the UI.
  - This subscribes the client to real-time `POSITION` updates for the specified flight.
  - ATC clients may track any active flight. Passenger clients may only track a flight they are booked on; the server must return an error if they attempt to track another flight.

8.5.2 Messages sent from the server to the Angular client

- `BOARDING_CALL` — Pushed by the server to all Passengers booked on a flight immediately after the ATC dispatches it.
  - The Passenger's UI must display this notification and present a "Confirm Boarding" button. Clicking that button sends a `BOARD` message back to the server.
- `POSITION` — Pushed by the server to all clients subscribed via `TRACK` on each animation step.
  - the current longitude and latitude of the aircraft.
  - When `progress` reaches its destination, the server must update the flight status to `Landed` via the `UpdateFlightPosition` API endpoint and send a final notification to all subscribed clients.

8.6 Lost Socket Handling

The server needs to account for lost sockets (when a client disconnects unexpectedly, e.g. the browser tab is closed). These lost sockets should be closed to allow for new connections. Consider the following:

- If the disconnected user was a **Passenger**, they will simply have to reconnect to the server.
- If the disconnected user was an **ATC**:
  - The flight animation must continue uninterrupted — the server does not depend on the ATC being connected.

  - All connected Passengers are notified that the ATC connection was briefly lost but that their flight is unaffected.
  - The ATC can reconnect and resume tracking.

8.7 General Server Behaviour

- The server should continuously interpolate and broadcast the position of all active aircraft.
- The server should notify Passengers when their flight has been dispatched.

You have two options for how you update the database with flight positions. *There is no incorrect answer here, but you need to motivate your choice.*

- **Every time there is an update** — Call `UpdateFlightPosition` on every animation tick.
- **On an interval** — Call `UpdateFlightPosition` only at a fixed interval, keeping the latest position in server memory between writes.

Write a short explanation of your choice and why you believe it is the better option in a `README.txt` included with your ClickUP submission.

8.8 Credentials

Since Wheatley is password-protected, you will need to include your login details in the URL as follows:

`username:password@wheatley.cs.up.ac.za/uXXXXXXXX/path_to_api`

It is your responsibility to keep your login details as safe as possible. It is recommended that you store your username and password in a configuration or environment variable rather than hardcoding them throughout your code. **Bonus marks may be given depending on how secure your solution is.**
NB: Ensure that you do not submit your code with your passwords included in your clickUP submission.

To test the functionality of your server, you may use `https://www.piesocket.com/websocket-tester` as the client before proceeding to Task 3.

Task 3: Angular Web Client ..... (50 marks)

For this task, you are required to develop an Angular 17 web application that connects to your locally running NodeJS WebSocket server. The client must use WebSockets for all real-time communication and Leaflet to display flight positions on an interactive map.

The client must run on localhost.

8.9 General Requirements

- The client must connect to your NodeJS WebSocket server.
- The user must log in using their username and password.
- After login, the interface must change dynamically based on the user's role.
- DOM manipulation should be handled within Angular components — do not reload the page after login.

8.10 Passenger View

After successful login as a Passenger:

- The Passenger can view a list of their booked flights.
- The Passenger can select a flight to track in real time.
- When an ATC dispatches a flight:
  - A boarding notification must appear immediately.
  - A visible 60-second countdown timer must be shown.
  - The Passenger must be able to click a `Board` button.
  - If not confirmed within 60 seconds, the Passenger is marked as a no-show.
- The Passenger may only track flights they are booked on.
- While tracking, the Passenger must see:
  - Live aircraft position updates.
  - Flight number.
  - Origin and destination.
  - Current status (`Scheduled`, `Boarding`, `In Flight`, `Landed`).
    - Flights must transition between statuses according to the following rules:
      - When an ATC dispatches a flight, its status changes from **Scheduled** to **Boarding**.
      - The 60-second boarding window begins immediately when the flight enters the Boarding state.
      - After the 60-second boarding window expires, the flight status must transition from **Boarding** to **In Flight**. The aircraft animation begins at this point.
      - When the animation progress reaches the destination, the flight status must transition from **In Flight** to **Landed**.

      These transitions must be enforced by the NodeJS server and reflected in the database via the appropriate API endpoints.

8.11 ATC View

After successful login as an ATC:

- The ATC can view all flights in the system.
- The ATC can select a `Scheduled` flight and dispatch it.

- The ATC can track any active flight.
- The ATC must see:
  - Full passenger list for the selected flight.
  - Boarding confirmations in real time.
  - Live aircraft position updates.
- If a Passenger fails to board within 60 seconds, the ATC must be notified.

8.12 The Map

The map is the central component of the client and must use Leaflet.

- The map must load with a default world view.
- All airports must be plotted using data from `GetAirports`.
- Airports must use distinct markers from aircraft markers.
- Aircraft must be represented by a moving marker.
- The aircraft must animate smoothly between origin and destination.
- The animation must reflect the scaled duration rule:
  - A flight with duration N hours must complete in N seconds.
- The client must update the aircraft position based solely on WebSocket messages from the server.
- The UI must show flight progress as a percentage (based on the `progress` value sent by the server).

8.13 WebSocket Message Handling

Your Angular client must properly handle at least the following message types:

- `DISPATCH`
- `POSITION`
- `SHUTDOWN`
- Error messages (403, 404, timeout, etc.)

If the socket disconnects:

- Display a visible connection error message.
- Prevent further interaction until reconnection.

8.14 System Flow Requirements

The Angular client must correctly reflect the full system flow:

1. ATC dispatches flight.
2. Passenger receives boarding notification.
3. Passenger confirms boarding.
4. Server begins aircraft animation.
5. Clients receive real-time position updates.
6. Flight reaches destination.
7. Status updates to `Landed`.

The UI must visually reflect all state transitions.

Note: Bonus marks may be given to students who add extra functionality. A maximum of 15 bonus marks may be awarded (up to 7 for standard extra functionality, but to achieve 15, you must add something extraordinary!). Bonus marks exceeding full marks will be capped.

Some ideas for bonus marks:

- Implement a live chat feature between Passengers and ATC.
- Add a flight trail/path animation showing where the aircraft has been.

Good Luck!